

Семинар 3 · закрепляет Лекцию 2

Три механизма модели в системном дизайне

Эмбеддинги · внимание · сэмплинг и токенизация — на шести рабочих задачах:
строим, правим, дорабатываем

Три блока, шесть задач

Блок 1 · Эмбединги

Выбор embedding-модели под свой корпус · дедупликация тикетов поддержки

Блок 2 · Внимание

Извлечение «кто кому платит» из договоров · ассистент юротдела и как измерить качество

Блок 3 · Сэмплинг и токенизация

Генератор карточек товара (описание + параметры) · токенизация ломает подсчёт и валидацию

Каждый кейс: контекст → решение и схема → усложнение из практики → разбор с примером



Блок 1

Эмбеддинги: сравнение и группировка по смыслу

Создаём `embedding`-поиск · дорабатываем поддержку

Какую embedding-модель взять под свой корпус

Строим семантический поиск по внутренней базе знаний: сотрудник спрашивает обычными словами — система находит документ по смыслу. Первое решение — какой моделью кодировать текст в векторы.

Корпус

свой, русскоязычный, с терминами предметной области

Выход

по запросу вернуть релевантные документы

Латентность

выдача в пределах ~300 миллисекунд

Смена модели дорогая

меняешь модель →
переиндексируешь весь корпус

Готовой разметки нет

таблицы «для запроса X релевантны Y, Z» не существует

За рамками

критерий выбора, список моделей, размерность — это решения, не данность

По какому критерию выбирать модель — и как проверить кандидатов на СВОИХ данных?

Как выбирать: сначала eval, потом всё остальное

Качество поиска — свойство пары «модель + твой корпус + твои запросы», а не самой модели. Меряется только на своих данных: насколько векторы модели сближают ТВОИ запросы с ТВОИМИ релевантными документами.

Как работает

- Отбери 3-4 кандидата разных классов по MTEB/ruMTEB (лидерборд эмбединг-моделей, ru- — русская версия) — как по списку, не как по ответу
- Собери мини-eval: 50-100 пар «запрос → релевантный документ» из своего корпуса + ловушки
- Прогони каждого кандидата на ОДНОМ наборе → recall@k и MRR в идентичных условиях
- Сопоставь качество с размерностью вектора: 384 vs 1024 → хранение и латентность поиска
- Выбирай по оси «качество на моих данных / стоимость», версию модели зафиксируй в конфиге

Кандидаты (по лидерборду)

Мини-eval (свои пары)

recall@k · MRR

Выбор: качество / стоимость

recall@k — доля запросов, где нужный документ попал в top-k. **MRR** (средний обратный ранг) — насколько высоко он стоит. **Размер eval-набора** — функция задачи (разнообразие запросов, цена ошибки), а не круглое «200»; даже 30 честных пар лучше выбора вслепую.

Три оси выбора — каждая проверяется тем же eval

Оси не сводятся к одной цифре: качество на своих данных, размерность вектора, стоимость и латентность. По каждой оси разные классы моделей дают разный компромисс.

Многоязычная / одноязычная

Многоязычная — широкое покрытие; русскоцентричная — богаче русский, уже покрытие.

- Многоязычные: e5, bge-m3 — усредняют качество по десяткам языков
- Русскоцентричные: RoSBERTa, GigaEmbeddings — кандидат на русском домене
- Проверяется: recall@k на русском eval

Доменная / общая

Доменная/дообученная под юр/мед/финтех vs крепкая общая база «из коробки».

- Доменная потенциально выше на своём домене
- Общей иногда достаточно и дешевле
- Проверяется: eval; «доменная=лучше» — гипотеза

Размерность

Компактная 384-768 против крупной 1536-3072 (флаг MTEB, вектор больше).

- Крупная: выше средний балл, дороже хранение и поиск
- Компактная: дешевле, ближе к бюджету 300 мс
- Проверяется: равный recall → бери меньшую

Не сравнивай модели по описанию — сравнивай по цифрам на своём корпусе. Крупная размерность линейно увеличивает объём векторной БД и время поиска на ВСЁМ индексе: если recall@10 у 768-мерной такой же, платить за 1536 незачем. Усечение вектора (матрёшка/MRL) проверяется тем же замером.

Лидер бенчмарка проседает именно на русском домене

В задокументированной боевой системе поиска по юридическим договорам топ-3 модели лидерборда MTEB на in-domain eval-наборе заняли 5-е, 7-е и 2-е места: модель, первая на общем бенчмарке, на реальном корпусе оказалась пятой — уступила менее «титулованным».

Средний балл бенчмарка усредняет десятки языков и доменов. «Лучшая по MTEB» и «лучшая на твоих данных» — разные модели; ось качества не одна.

Пайплайн выбора: лидерборд → eval → метрики → выбор

Лидерборд отбирает кандидатов, дальше решение принимает замер на своих данных.



- Лидерборд MTEB/ruMTEB — только источник списка кандидатов, не ответ
- Каждый кандидат прогоняется на ОДНОМ наборе в идентичных условиях
- Выбор по оси «качество на моих данных / стоимость»; версия победителя фиксируется — смена = переиндексация

Час eval против недель жизни с неудачным, дорого-переиндексируемым индексом.

Разбор · компоненты и комментарии

Ключевые компоненты

Список кандидатов

3-4 модели разных классов по MTEB/guMTEB как по списку; можно добавить BM25 как не-ИИ базлайн.

Мини-eval набор

Таблица «запрос | id документа | тип (обычный/ловушка)», 50-100 строк. Единственный арбитр качества.

Кодировщик (encoder)

Модель текст→вектор; важны режимы query/document и метрика/нормализация под обучение модели.

Индекс ближайших

pgvector / Qdrant / FAISS — хранит векторы корпуса, быстро возвращает top-k.

Конфиг с пиннингом

Зафиксированная версия модели + метрика + размерность — защита от тихого обновления.

Ключевые комментарии по теме

— Вектор отражает статистику того, что модель видела при обучении → «качество поиска» неотделимо от пары «модель + твои данные».

— similarity $\neq$ релевантность: высокое сходство у пар «включить/отключить» — граница самого метода, снимает её реранкер/гибрид, не смена модели.

— Грабли: метрика/нормализация $\neq$ обучению; одинаковое кодирование query/document; выбор по лидерборду без eval; «возьму побольше на всякий случай».

Разбор · как реализовать

Выбор embedding-модели = мини-eval на своих парах + осознанный размен размерности; лидерборд — только источник кандидатов, версия фиксируется в конфиге.

Как реализовать

- Собери 50-100 пар «запрос → релевантный документ» из своего корпуса + ловушки «вкл/выкл» — это твой eval
- Прогони кандидатов (многоязычную, русскоцентричную, крупную, BM25) на ОДНОМ наборе → recall@k и MRR
- Сопоставь качество с размерностью: если 768 даёт тот же recall, что 1536 — бери 768
- Проверь совпадение метрики (cosine/dot) и нормализации с тем, как обучалась модель; зафиксируй версию

V Час eval против недель с плохим индексом

Финальную проверку делай на выборке своих документов и запросов: час работы против недель с неудачным индексом. Лидер MTEB ≠ лидер на твоём домене.

Схлопнуть дубли в потоке тикетов поддержки

Служба поддержки платёжного сервиса принимает поток обращений; одна проблема приходит много раз, сформулированная десятками способов. Хотим объединять похожие автоматически.

Поток

порядка сотен обращений в день,
непрерывно

Дубли

одна проблема — разные
формулировки, часто без общих
слов

Есть структура

тип карты, код ошибки,
идентификатор сервиса

Разметки почти нет

готового набора «дубль / не дубль»
под наш поток не существует

Цена ошибки несим.

склеить разное = спрятать и
потерять инцидент

Решение сразу

в момент поступления, не батчем
раз в сутки — иначе оператор
разбирает дубль вручную

Как находить и объединять дубли — и где поставить страховку от ложного склеивания?

Как устроен дедуп — и зачем структурный гейт

Дедуп по эмбедингам: текст тикета кодируем в вектор, близкие по косинусу — кандидаты в один кластер по порогу τ . Структурный гейт — жёсткие поля, обязанные совпасть, чтобы объединение было допустимо.

Как работает

- Векторизуем текст тикета → близкие формулировки дают близкие векторы, даже без общих слов
- Порог τ по косинусу: пары с близостью $> \tau$ — кандидаты на объединение («похоже?»)
- Структурный гейт: объединяем ТОЛЬКО если совпали ключи — тип карты, код ошибки, сервис («то же ли?»)
- Не совпали ключи при высокой близости → не мержим, помечаем на ручной разбор

Тикет → вектор

Близость $> \tau$?

Совпали структурные ключи?

Объединить / отдельно

Порог τ — не константа из примера, а отдельный шаг настройки: его выставляют по размеченным парам через кривую *precision-recall*. Размер размеченного набора — функция задачи (частота потока, цена ошибки, разнообразие формулировок), а не круглое «200».

«Visa» и «Mastercard»: похоже по вектору, но не одно и то же

«Оплата не проходит картой Visa» и «оплата не проходит картой Mastercard» почти идентичны по вектору (косинус 0.95+): отличается одно слово из шести. Но это разные инциденты — разные эквайринги, разные шлюзы, разные причины. Чистый порог близости их склеит и спрячет один инцидент внутри другого.

Эмбеddинг кодирует ТЕМУ и систематически теряет различающую деталь. Крутить τ вверх бесполезно: любой τ , разделяющий Visa/Mastercard, выбросит массу настоящих дублей.

Гибрид «близость + структурные ключи»

Текст решает «похоже», детерминированные поля решают «то же ли это». ANN — приближённый поиск ближайших соседей (быстро находит похожие векторы).



- Similarity набирает кандидатов дешево и широко (инкрементально, через ANN-индекс ближайших соседей)
- Структурный гейт по обязательным полям отсеивает ложные склейки точно и безопасно
- Высокая близость + разошедшиеся ключи → не мержить, на ручной разбор, не молча в новый кластер

В одиночку не закрывает ни один механизм: близость склеивает near-duplicate, ключи без близости не ловят дубли.

Разбор · компоненты и комментарии

Ключевые компоненты

Нормализатор / извлекатель полей

Тикет из любого канала → чистый текст + структурные поля (regex/парсер). От него зависит надёжность гейта.

Embedding-модель

Текст → вектор (выбрана в кейсе 1.1); метрика и нормализация под её обучение.

ANN-индекс

FAISS / Qdrant / pgvector — соседи инкрементально, без квадратичного пересчёта.

Порог близости τ

Единственный «мягкий» параметр; по кривой precision-recall под фиксированную precision.

Структурный гейт

Детерминированное «все ключи совпали» — несёт различающую деталь, которую вектор теряет.

Очередь ручного разбора

Спорные near-duplicate человеку — страховка от потери инцидента + источник новых ловушек.

Ключевые комментарии по теме

- Эмбеddинг кодирует «о чём» текст и теряет различающую деталь при малом лексическом различии — косинус даёт кандидатов, не тождество.
- Главная грабля — разводить near-duplicate поднятием τ : любой достаточный τ выбросит настоящие дубли. Различающую деталь ловит структура.
- Несимметричная цена ошибки → приоритет precision, не баланс; недобранный recall добираем гибридом и ручным разбором.

Разбор · как реализовать

Дедуп = близость по вектору + обязательные структурные ключи: текст решает «похоже», ключи решают «то же ли это». Спорное — на ручной разбор, не молча.

Как реализовать

- Извлекаем структурные поля (тип карты, код ошибки, сервис) — regex/парсер из машинных источников
- Выставляем τ отдельным шагом: размечаем пары дубль/не дубль (много ловушек), строим precision-recall, берём τ под нужную precision
- Кандидат на merge = близость $> \tau$ И совпадение обязательных ключей
- Спорные (близость высокая, ключи разошлись) не мержим — на ручной разбор; на релизе меряем долю ложных склеек

Уверенные ложные склейки (near-duplicate)

«Pure text similarity produces confident false merges»: близость по смыслу не гарантирует тождества — различающую деталь ловит структурный гейт, не порог.



Блок 2

Внимание: что модель понимает в тексте

Создаём извлечение · дорабатываем ассистента

Извлечь «кто, кому, сколько и когда платит»

В юротделе накопились сотни договоров. Пока платёжные обязательства живут только в тексте, ими нельзя пользоваться как данными. Задача — превратить свободный текст в структуру платежей.

Вход

договоры разных форматов: docx, живые PDF, сканы

Извлекаем

плательщик, получатель, сумма, срок, условие

Денежные поля критичны

ошибка в сумме или направлении = неверная проводка

Всё вручную — нет

сотни договоров, тысячи полей не проверить руками

Аудируемость

по каждому значению видеть, из какого места текста оно взято

За рамками

критерии достаточности и где главная сложность — предмет разбора

Каким должен быть пайплайн — и где ставить проверку критичных полей?

Значение + span-обоснование + гейт уверенности

Внимание (attention) надёжно разрешает «кто кому», пока в тексте есть однозначный сигнал, и системно ошибается, когда однозначного ответа нет. Значит: массовое извлечение — модели, критичные поля — под не-модельную проверку.

Как работает

- По КАЖДОМУ полю модель возвращает тройку: значение + evidence_span (дословная цитата) + флаг уверенности
- span проверяется как подстрока источника: не нашёлся → сигнал выдумки, поле спорно
- Форму фиксирует structured output: модель обязана вернуть ответ по JSON-схеме, strict = не вернёт поле вне схемы (параметр response_format) — гарантирует форму, НЕ истинность
- Детерминированные кросс-проверки кодом: сумма прописью = числом, даты валидны, стороны однозначны
- Гейт: нет span / низкая уверенность / провал проверки / обоснование-местоимение → к человеку

Договор

LLM-извлечение + span

Кросс-проверки (код)

Гейт уверенности

Человек на спорных

Уверенность бери из объективных сигналов (наличие span, кросс-проверки, лог-вероятности), а не из вопроса модели «ты уверена?» — вербальная самооценка измеряет натренированность звучать уверенно, а не реальную надёжность.

«...если он предоставит гарантию» — кто «он»?

«Заказчик выплачивает Исполнителю аванс, если он предоставит гарантию.» Кто должен дать гарантию? Грамматически оба — мужского рода, согласование не помогает. Winobias (бенчмарк кореференции, 3160 предложений): смена одного слова переворачивает референт, а системы связывают местоимения с про-стереотипными сущностями точнее на 21.1 пункта F1 (метрика точности, 0-100).

Механизм внимания опирается на статистическую корреляцию слов, а не на причинно-следственную связь. На неоднозначной кореференции он ошибается системно и в предсказуемую сторону — уверенным тоном.

LLM-извлечение с верификацией критичных полей

Модель — быстрый первый проход; проверка — там, где цена ошибки высока.



- Мусорный вход (плохой OCR) не пускается дальше — порог качества текста до извлечения
- Детерминированные проверки ловят несходящиеся суммы и невалидные даты без модели
- Нет span / низкая уверенность / обоснование-местоимение → на человека; деньги не идут в прод мимо гейта

Автоматизируем массу, страхуем критичное — не «всё или ничего».

Разбор · компоненты и комментарии

Ключевые компоненты

Нормализатор входа

OCR (Tesseract/Yandex Vision/ABBYY) + pdfplumber/PyMuPDF + порог качества текста. Модель получает текст, не шум.

LLM-экстрактор

Топовая модель со structured output (json_schema, strict), стабильная часть промпта в начало (кэш префикса).

Слой кросс-проверок

num2words/парсер прописи, dateutil/workalendar, справочники валют и сторон, pydantic. Ошибки без модели.

Гейт уверенности

span найден $\wedge$ confidence $\geq$ порог $\wedge$ проверки пройдены $\wedge$ обоснование не местоимение. Делит авто/человек.

Очередь на человека

Таблица спорных полей с цитатой и договором — сфокусированная проверка 5-15% полей.

Eval-набор Winograd-типа

Спорные случаи с эталонами — меряет кореференцию на каждом релизе, ловит регресс.

Ключевые комментарии по теме

- Внимание опирается на статистическую корреляцию слов, а не на причинность: на однозначном — отлично, на неоднозначном системно выбирает частый вариант.
- span, а не «ты уверена»: уверенный тон не коррелирует с наличием однозначного ответа; цитата либо есть в тексте, либо нет.
- Схема гарантирует форму JSON, но не истинность значения — истинность дают кросс-проверки и гейт, а не строгая схема.

Разбор · как реализовать

Извлечение — модель; разрешение неоднозначной кореференции в денежных полях — под верификацию и правила. Форму даёт схема, истинность — код и гейт.

Как реализовать

- Промпт возвращает по каждому полю: значение + span-обоснование из текста + флаг уверенности
- Детерминированные кросс-проверки: сумма прописью = числом, даты валидны, стороны разрешаются в словарь из шапки
- Гейт: поле без однозначного span, с низкой уверенностью или обоснованием-местоимением → на ручную проверку
- Держи eval-набор из неоднозначных договоров (Winograd-типа), меряй долю верных референтов и долю эскалаций

Х Галлюцинации даже с RAG (юр/фин домен)

Стэнфорд, 2024: инструменты с привязкой к источникам галлюцинировали — Lexis+ AI 17%, Westlaw 33%, чистый GPT-4 43%. Денежные поля без верификации недопустимы.

«Отвечает неточно — сделайте качественнее»

Год работает чат-ассистент юротдела на общей модели. Заказчик приходит с расплывчатым «сделайте, чтобы отвечал качественнее»: ответы звучат уверенно, но факты и ссылки на нормы регулярно неточны.

Что есть

работающий ассистент на общей модели, базой норм не пользуется

Жалоба

уверенный тон, но факты и ссылки на нормы неточны

Метрики нет

«качество» существует только как ощущение — ни числа, ни точки отсчёта

Есть база

актуальные регламенты и нормы в машиночитаемом виде

Есть логи

тысячи реальных вопросов юристов за год

Цена ошибки

неверная ссылка на норму → юридический риск

Как поднять качество — и как доказать, что подняли, а не просто «звучит солиднее»?

Сначала метрика и базлайн, потом изменения

Нельзя улучшить то, что не измеряешь. Генеративная система коварна: её выход всегда выглядит как ответ, и на глаз верный ответ и уверенная выдумка неразличимы. Нужен воспроизводимый измеритель — golden set.

Как работает

- Собери golden set: 50-100 реальных вопросов из логов с эталонными ответами и верными ссылками
- Три метрики: фактическая точность · корректность ссылки на норму · доля ответов с проверяемым источником
- Считай автоматом: смысл — LLM-as-judge (оценка ответа более сильной моделью по рубрике, калиброванный), ссылку — точным совпадением строки (код, не судья)
- Замерь базлайн текущего ассистента на наборе — это точка отсчёта
- Меняй по одной вещи и мерь на том же наборе; изменение принимается только при росте метрики

Вопрос

Ответ ассистента

Сверка с эталоном / нормой

3 метрики

Golden set — обычная таблица «вопрос | эталонный ответ | верная ссылка». Размер — функция задачи (разнообразие трафика, цена ошибки, число тем), а не круглое «200»; критерий достаточности — набор перестал приносить новые типы ошибок.

«Ты — опытный юрист» на метрике не двигается

Частая первая попытка — добавить в промпт роль «Ты — опытный юрист с 20-летним стажем». Zheng et al., 2024: 162 роли-персоны на 2410 фактических вопросах, четыре семейства моделей — добавление роли НЕ повышает точность; эффект персоны по сути случаен. На eval-наборе ответы увереннее по форме, но не вернее по сути.

Роль — тот же attention-механизм, что разрешал кореференцию: токены роли смещают выбор в сторону стиля, но не добавляют знаний. Форма и знания — разные оси. Точность даёт grounding, не «титул» в промпте.

Разбор · компоненты и комментарии

Ключевые компоненты

Golden set

Таблица «вопрос | эталон | верная ссылка» из реальных логов. Единственная опора, отличающая улучшение от ухудшения.

Сборщик набора

Кластеризация логов по эмбедингам + отбор для равномерного покрытия тем. Меряет реальный трафик.

Прогонщик (eval runner)

Ragas / DeepEval / Promptfoo — гоняет версию по набору, собирает три метрики.

LLM-судья

Более сильная модель, оценивает смысл по рубрике, $T=0$, калибруется ручной выборкой.

Детерминированная сверка

Код: нормализация и сравнение ссылок, проверка наличия цитаты. Без галлюцинаций и дрейфа.

RAG-grounding на нормы

Поиск фрагментов базы + ответ по ним с обязательной цитатой. Реальный рычаг точности (по метрике).

Ключевые комментарии по теме

— «Поднять качество» — сначала задача измерения, потом изменения: без числа любая правка вслепую, а уверенный тон и выдумка на глаз неразличимы.

— Форма и знания — разные оси: роль двигает форму (attention-вес токенов роли), знания приходят из данных в контексте (grounding).

— grounding — рычаг, не выключатель: даже RAG-инструменты дают 17-33% ошибок; изменение принимают по метрике, судью калибруют, ссылки сверяют кодом.

Разбор · как реализовать

Сначала **golden set** и базлайн; качество поднимает **grounding** на актуальные нормы, проверенный по метрике — не формулировка роли. Меняй по одной вещи за раз.

Как реализовать

- Собери **golden set** (вопрос/эталон/ссылка) из реальных логов, замерь базлайн текущего ассистента
- Проверь гипотезу «роль»: с ролью и без — метрика та же (это успех eval, а не провал)
- Добавь RAG на нормативную базу с обязательной цитатой; точность и **grounding rate** растут — принимаем
- Автопроверка: точность — LLM-судья (калиброван), ссылка — точное совпадение строки; изменение только при росте метрики



Роль вместо измеримого качества

Zheng et al., 2024: 162 роли-персоны, 2410 вопросов — роль-в-промте улучшает форму, не фактологию. Рост точности даёт **grounding**, и это видно на eval, а не на глаз.



Блок 3

Сэмплинг и токенизация: как модель порождает и видит текст

Подбираем режим генерации · упираемся в границу подсчёта

Карточки товара: описание и структурные параметры

Маркетплейс заводит карточки тысячами в день. У карточки две принципиально разные группы полей: маркетинговое описание (свободный текст) и структурные атрибуты (машиночитаемые поля).

Объём

тысячи карточек в день, ручное написание нереально

Описание

живой связный текст, единый бренд-голос, не «под копирку»

Атрибуты

габариты, вес, категория из справочника — строго определённая форма

Регламентные поля

состав, срок годности, гарантия — источник истины ГОСТ/паспорт

Цена ошибки разная

маркетинг — дешёвая и обратимая;
регламент — ответственность перед регулятором

Какой режим генерации ставить — и один ли на все поля?

Один промпт, три температуры: ось разброса, не «цветистости»

Один и тот же фактический вопрос, три температуры, повторные прогоны на каждой. Ось — «предсказуемо/повторяемо ↔ случайно/разнообразно и рискованно».

T = 0 (повторяемо)

Повторные прогоны — идентичные ответы. Выбор почти детерминирован.

- Предсказуемо и повторяемо
- Классификация, извлечение, атрибуты
- Разброса нет по построению

T ≈ 0.7 (рабочий разброс)

Формулировка гуляет между прогонами, но факт и связность держатся.

- Разнообразие формы без потери сути
- Маркетинг, объяснения
- Рабочий выбор для описаний

T = 1.3+ (риск)

Ответ расползается: связность теряется, лезут фактические ошибки и вставки.

- Черновики идей, мозгошторм
- Не для фактов
- Риск несвязности и выдумки

Мини-эксперимент: 20-30 карточек разных категорий, один промпт при 0.4 / 0.7 / 1.0 / 1.3 по 3 прогона, оценка по «разнообразие формулировок без фактических ошибок и в бренд-тоне». Рабочая зона обычно 0.6-0.8; выбранное T фиксируется в конфиге, не дефолт провайдера.

Температура задаёт разброс; схема задаёт форму, не истину

Температура (temperature) — одна ручка: насколько случаен выбор следующего слова. Низкая → выбор предсказуем и повторяем (модель почти всегда берёт самый вероятный вариант); высокая → выбор случайнее и разнообразнее, но рискованнее. Порядок «что вероятнее» и сами факты она не меняет.

Как работает

- Низкая T → почти всегда самый вероятный вариант; предсказуемо и повторяемо (при $T=0$ — `argmax`)
- Высокая T → редкие варианты получают шанс; разнообразнее, но лезут несвязность и выдумка
- $T \neq$ обрезка: `top-k` / `top-p` ОТСЕКАЮТ маловероятные варианты (по числу / по массе), T лишь меняет случайность выбора
- `structured output (json_schema, strict)` — модель обязана вернуть ответ по JSON-схеме; `strict` = не вернёт поле вне схемы (параметр `response_format`)
- Схема гарантирует ФОРМУ (числа числами, категория из справочника), но НЕ истинность значения

Выбор след. слова

Температура: случайность

`top-k/top-p`: отсечь редкое

Схема: форма, не истина

Правильная ось температуры — «предсказуемо/повторяемо ↔ случайно/разнообразно и рискованно», НЕ «скучно ↔ ярко». Высокая T не делает стиль цветистее — она делает выбор слов более случайным. Тон бренда задаётся промптом и примерами, не температурой.

Регламентные поля: ни одна температура не спасает

Состав, противопоказания, гарантийные условия, «соответствует ГОСТ...» — те же карточки, но вариативность недопустима, текст должен точно повторять источник. Air Canada (2024): чат-бот «дописал» несуществующую политику возврата, трибунал признал компанию ответственной — за сгенерированное утверждение о факте отвечает компания.

Температура управляет разбросом, а не достоверностью. Даже $T=0$ делает вывод повторяемым, но не истинным — модель может стабильно повторять неверное. Для регламентных полей нужен не «правильная T », а подстановка из проверяемого источника.

Пайплайн с тремя ветками: два запуска + подстановка

Разные группы полей — разные режимы генерации; регламентные поля идут мимо модели.



- Запуск А — свободный текст, температура подобрана на батче, вход — только известные факты
- Запуск Б — structured output (json_schema, strict) + T=0 + поле evidence на каждое значение
- Регламентные поля (состав, срок, гарантия) — подстановка из паспорта/ГОСТа БЕЗ генерации; подозрительное — человеку

Один вызов на всё — ложная экономия одной строки кода ценой худшего результата в обеих группах полей.

Разбор · компоненты и комментарии

Ключевые компоненты

Классификатор полей

Детерминированный слой (правила/конфиг): генерить свободно, структурно или брать из источника. Реализует «разные поля — разные режимы».

Запуск А (описание)

Свободный текст: промпт с бренд-тоном + few-shot, $T \approx 0.6-0.8$ из конфига, вход — только известные факты.

Запуск Б (атрибуты)

structured output (response_format/json_schema/strict), $T=0$, схема с полем evidence. Валидная форма.

Валидатор значений

Код после Б: диапазоны, единицы, evidence во входе, категория из справочника. Истинность, которую схема не даёт.

Подстановщик регламента

Шаблон + источник (паспорт, ГОСТ, справочник гарантий). Дословная подстановка без генерации.

Конфиг

Версия модели, температуры по типам полей, версия схемы — единая точка осознанных решений.

Ключевые комментарии по теме

- Температура = ось разброса, не стиля и не достоверности: высокая T повышает случайность выбора токена, на фактах ведёт к выдумке.
- Схема гарантирует форму, не истину значения: `weight_kg=5.0` может быть выдуман → обязательны валидация значений и поле evidence.
- $T=0 \neq$ истинность (и не полный детерминизм на облаке): регламентные поля не спасает никакая T , их источник истины — паспорт/ГОСТ, не модель.

Разбор · как реализовать

Разные поля — разные режимы: $T \approx 0.7$ для маркетинга, `structured output` + $T=0$ для атрибутов, подстановка из источника для регламентных полей. Форму даёт схема, истину — код и источник.

Как реализовать

- Раздели поля карточки классификатором: маркетинговые (генерим), структурные (структурно), регламентные (подставляем)
- Маркетинг: $T \approx 0.6-0.8$, подобранная на батче 20-30 карточек; бренд-голос — через промпт и `few-shot`, не температуру
- Атрибуты: `structured output (json_schema, strict)`, $T=0$, поле `evidence`; после — валидация значений кодом
- Регламент: шаблон с подстановкой из проверенного источника, без свободной генерации на любой T

Structured output как надёжная форма

OpenAI: `structured outputs` в `strict` дают 100% соответствия схеме против $\sim 86\%$ у `function calling` — модель физически не выпустит токен вне схемы.

Посимвольные фичи: пароль, карта, лимит, подсчёт

Веб-сервис с регистрацией и приёмом платежей. Набор мелких фич, работающих на уровне отдельных символов. На планёрке звучит: «у нас же есть модель — попросим её, она умнее регэкспов».

Фича 1 · пароль

длина ≥ 12 , классы символов, не 3+ одинаковых подряд + список нарушений

Фича 2 · карта

видны последние 4 цифры, остальные звёздочки (требование PCI DSS)

Фича 3 · обрезка

ровно под лимит символов (SMS-сегмент, поле БД), не разорвав слово

Фича 4 · подсчёт

число вхождений символа, длина, артикул ровно 8 символов

Точность обязательна

код на каждом запросе без человека; ошибка на символ = утечка или отказ формы

Чем реализовать каждую фичу и почему — моделью, кодом или комбинацией?

Почему модель структурно не считает символы

Токен — идентификатор из словаря модели. Текст режется на токены (частые подпоследовательности) и превращается в числа; дальше модель работает ТОЛЬКО с числами, не с буквами. Границы токенов не совпадают с границами букв.

Как работает

- «strawberry» — это 3 токена [st][raw][berry], не 10 букв; «клубника» — тоже 3 токена
- Внутри токена [raw] НЕТ отдельного представления буквы «r» на позиции 1 — усреднённая статистика контекстов
- Словарь строится алгоритмом BPE (Byte-Pair Encoding — как режется текст на токены) один раз до обучения и не меняется
- компромисс ради экономии внимания
- Посимвольное представление удлинило бы вход в 3-5 раз, а стоимость внимания растёт квадратично
- Значит подсчёт символов, длина, посимвольная резка ненадёжны — и это НЕ «починят в новой версии»

Строка «strawberry»

BPE-нарезка на токены

[st][raw][berry] = 3 числа

Букв как единиц нет

Jagged intelligence («рванный интеллект»): одна модель решает олимпиадную задачу и проваливает подсчёт букв в 9-буквенном слове. Успех на впечатляющей задаче НЕ свидетельство надёжности на «простой». «Написала SQL — уж символы посчитает» — ошибочная экстраполяция.

Патч ≠ навык: strawberry чинят, cranberry ломается

Гонка точечных патчей: GPT-5.2 отвечал «две r» в strawberry; GPT-5.5 чинит strawberry, но на «сколько r в cranberry» отвечает «две» (правильно три); GPT-5.6 чинит cranberry. Каждый вирусный вопрос дожидается своего исправления, но класс задачи не решается.

Вирусное прохождение конкретного примера ≠ навык. Модель проходит ИЗВЕСТНЫЕ примеры (дообучены или эвристика), а твой артикул или пароль ей неизвестны. Замени слово на редкое, невирусное — провал возвращается.

Детерминированное — коду, смысловое — модели

Раздели задачу по природе результата: точный результат на уровне символов или понимание смысла.



- Точный результат на уровне символов/длины/формата/чисел → код: len, регэксп, срезы, count, форматтеры
- Понимание смысла (это спам? тональность? перефразируй?) → модель, здесь у неё нет конкурента
- Смешанное (сократи под 160 символов сохранив мысль): модель — смысл, код — точная длина (truncate поверх)

Код: микросекунды, бесплатно, 100% воспроизводимо, тестируемо. API модели: десятки-сотни мс, оплата токенов, вероятностный результат.

Разбор · компоненты и комментарии

Ключевые компоненты

Классификатор природы задачи

Не код, а решение инженера/чек-лист: «результат на уровне символов и точный → код; смысл → модель». До реализации.

Символьный код

len (длина), регэксп (классы, формат, позиции), срезы (s[:n], s[-4:]), count, форматтеры. Микросекунды, тестируем.

Модель (LLM)

Только смысл: классификация, извлечение сути, перефразирование, понятная формулировка ошибки. Никогда — точный счёт/длина/маскирование.

Обрамление вывода модели

Для смешанных задач: truncate()/validate() поверх ответа модели гарантирует точную форму.

Юнит-тесты границ

Порог ± 1 , пустая строка, пробелы, эмодзи/не-ASCII, длинная строка. Ловят ошибки на «единицу».

Визуализатор токенизации

tiktokenizer.vercel.app / tiktoken — инструмент понимания и демонстрации, не рантайм.

Ключевые комментарии по теме

- Внутри токена нет надёжного представления «какая буква на какой позиции» → счёт, длина, резка, валидация формата — структурно не задача модели.
- Пара 3.1/3.2 — одна природа модели с двух сторон: знать, где механизм работает на тебя (генерация), и где против тебя (подсчёт).
- Грабли: экстраполяция от сложного к «простому» (jagged); «ответила на моём тесте» (проверь редкое); чувствительные данные в API; нет тестов границ.

Разбор · как реализовать

Символьное и детерминированное делает код; модель — на смысл, не на подсчёт. Если без модели никак — она делает смысл, а точную форму гарантирует код-обрамление.

Как реализовать

- Пароль — `len` + регэксп по классам символов; карта — срез строки (`d[-4:]` + звёздочки), локально, ни байта в API
- Обрезка под лимит — срез + `len`; подсчёт и формат — `count` и `re.fullmatch`; всё детерминированно и в микросекундах
- Смешанная задача — модель делает смысл, код гарантирует форму: `truncate(вывод_модели, 160)` поверх
- Заведи юнит-тесты границ: порог ± 1 , пустая строка, пробелы, эмодзи/не-ASCII, длинная строка



Код на месте, модель на месте

Промышленные системы валидации форм, маскирования и обрезки НЕ используют LLM для символьной части — это регэксп, `len`, срезы, покрытые тестами. LLM включается только на смысл.

Что унести

- Эмбеддинги: выбор модели решает мини-eval на своих данных (recall@k/MRR), не место в бенчмарке; «похоже» $\neq$ «то же» → структурные ключи.
- Внимание: корреляция, не причинность → модель извлекает + обосновывает span, правила и человек проверяют критичное; качество двигает grounding, не роль.
- Сэмплинг и токенизация: температура задаёт разброс (не стиль, не истину), схема — форму (не истину); подсчёт символов — не ниша модели, его делает код.

Один принцип на все три: механизм — не «фича модели», а граница, которую инженер знает в обе стороны — использует силу и страхует слабость. Дальше — Лекция 3: как из этих механизмов собираются архитектуры (RAG, агент).